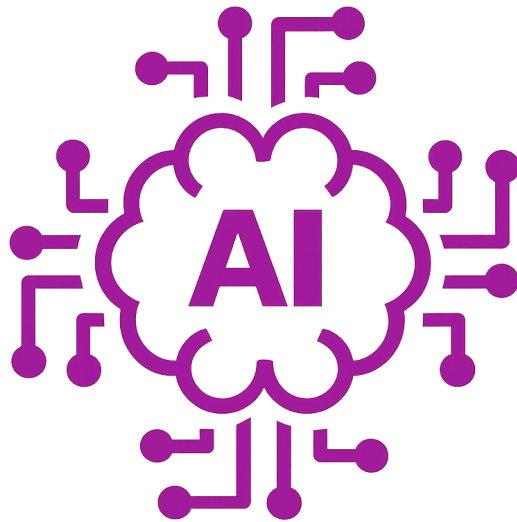


# ML / DL : PBL 02

« Обучение и предсказание регрессионной модели на  
данных о ценах жилья »



SK Shieldus

Rookies, 30-й набор (личный номер 14)

**Ким Джэ-Ук**

## Решение PBL 02

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# загружаем данные для обучения
df = pd.read_csv("20260112_153749_train.csv")
```

```
# разведочный разбор, посмотрим, из чего состоят данные
print(f"В этой таблице число строк {df.shape[0]} и число столбцов {df.shape[1]}")
print("-"*30)
# смотрим, какие именно есть столбцы, возьмём для примера десять
print("Возьмём для примера десять столбцов\n"
      f'{', '.join(df.columns.values.astype(str)[:10])}')"
```

В этой таблице число строк 1460 и число столбцов 81

-----

Возьмём для примера десять столбцов

Id, MSSubClass, MSZoning, LotFrontage, LotArea, Street, Alley, LotShape, LandContour, Utilities

```
print("-"*5, " Первые десять значений столбца 'LotFrontage' ", "-"*5)
print(df["LotFrontage"].head(10))
```

----- Первые десять значений столбца 'LotFrontage' -----

```
0    65.0
1    80.0
2    68.0
3    60.0
4    84.0
5    85.0
6    75.0
7     NaN
8    51.0
9    50.0
```

Name: LotFrontage, dtype: float64

**По выводу выше можно составить общее представление о данных в столбце 'LotFrontage'.**

```

# считаем общее число пропусков по каждому столбцу
# сначала хотел посмотреть только пятнадцать переменных
# top15_null_col = null_count_by_col.sort_values(ascending=False).head(15)
# print(top15_null_col) # пятнадцать столбцов по убыванию числа пропусков

# но решил, что лучше просмотреть все переменные, и вывел их полностью
null_count_by_col = df.isnull().sum()
all_null_col = null_count_by_col.sort_values(ascending=False)

# берём HTML, чтобы таблица выглядела опрятнее, чем при print
from IPython.display import HTML
# сохраняем прежние названия строк, которые лежали в индексе
html_table = all_null_col.reset_index( drop = False )
html_table.columns = ["Feature" , "Число"]
html_table["Доля"] = (html_table["Число"] / len(df)) * 100

total_row_num = len(html_table)
half_row_num = (total_row_num + 1) // 2
# делим строки на левую и правую часть, потому что слишком вытянутая
# вниз таблица зря тратит место по ширине, это неэкономно
# единицу в скобках прибавляем, чтобы при нечётном числе строк
# в левой части оказалось на одну строку больше
# берём iloc, чтобы выбирать строки по порядку
left = html_table.iloc[:half_row_num].reset_index(drop = True)
right = html_table.iloc[half_row_num:].reset_index(drop = True)

left.insert(0, "Index", range(1, len(left) + 1))
right.insert(0, "Index", range(len(left) + 1, len(left) + len(right) + 1))

# проходим по левой и правой части и убираем запись вида ".0",
# а то, что нельзя превратить в число, например NaN, оставляем как есть
for side_table in (left, right) :
    # столбцов всего 81, поэтому справа последняя клетка остаётся пустой
    # и Index выводится как ".0", закрепляем целый тип Int64 и убираем это
    side_table["Index"] = pd.to_numeric(side_table["Index"],
                                         errors="coerce").astype("Int64")
    side_table["Число"] = (pd.to_numeric(side_table["Число"],
                                         errors = "coerce").astype("Int64"))
    side_table["Доля"] = pd.to_numeric(side_table["Доля"],
                                         errors="coerce").round(2)

# axis = 1 означает, что части ставятся рядом по горизонтали
merged = pd.concat([left, right], axis = 1)
merged_display = merged.astype(object).where(merged.notna(),"-")

# как уже было видно на первой странице, всего в наборе 81 столбец,

```

```
# поэтому в левой и правой половине таблицы число строк не совпадает
# при настройках по умолчанию внизу правой таблицы появилась бы
# запись NaN, чтобы сравняться с левой половиной
# тогда последнюю клетку легко принять за пропуск, и чтобы этого
# не случилось, ставим вместо неё знак -
# то есть внимание, и Feature, и число наблюдений показаны знаком -,
# и это не пропуск, а неизбежная пустота из-за нечётного числа (81)
```

```
HTML(merged_display.to_html(index = False, border = 1))
```

| Index | Feature      | Число | Доля  | Index | Feature       | Число | Доля |
|-------|--------------|-------|-------|-------|---------------|-------|------|
| 1     | PoolQC       | 1453  | 99.52 | 42    | Exterior2nd   | 0     | 0.0  |
| 2     | MiscFeature  | 1406  | 96.3  | 43    | Exterior1st   | 0     | 0.0  |
| 3     | Alley        | 1369  | 93.77 | 44    | RoofMatl      | 0     | 0.0  |
| 4     | Fence        | 1179  | 80.75 | 45    | RoofStyle     | 0     | 0.0  |
| 5     | MasVnrType   | 872   | 59.73 | 46    | YearRemodAdd  | 0     | 0.0  |
| 6     | FireplaceQu  | 690   | 47.26 | 47    | OverallQual   | 0     | 0.0  |
| 7     | LotFrontage  | 259   | 17.74 | 48    | YearBuilt     | 0     | 0.0  |
| 8     | GarageQual   | 81    | 5.55  | 49    | OverallCond   | 0     | 0.0  |
| 9     | GarageFinish | 81    | 5.55  | 50    | BsmtHalfBath  | 0     | 0.0  |
| 10    | GarageType   | 81    | 5.55  | 51    | BsmtFullBath  | 0     | 0.0  |
| 11    | GarageYrBlt  | 81    | 5.55  | 52    | GrLivArea     | 0     | 0.0  |
| 12    | GarageCond   | 81    | 5.55  | 53    | LowQualFinSF  | 0     | 0.0  |
| 13    | BsmtFinType2 | 38    | 2.6   | 54    | 2ndFlrSF      | 0     | 0.0  |
| 14    | BsmtExposure | 38    | 2.6   | 55    | 1stFlrSF      | 0     | 0.0  |
| 15    | BsmtCond     | 37    | 2.53  | 56    | CentralAir    | 0     | 0.0  |
| 16    | BsmtQual     | 37    | 2.53  | 57    | HeatingQC     | 0     | 0.0  |
| 17    | BsmtFinType1 | 37    | 2.53  | 58    | BsmtFinSF2    | 0     | 0.0  |
| 18    | MasVnrArea   | 8     | 0.55  | 59    | Fireplaces    | 0     | 0.0  |
| 19    | Electrical   | 1     | 0.07  | 60    | TotRmsAbvGrd  | 0     | 0.0  |
| 20    | Condition2   | 0     | 0.0   | 61    | KitchenQual   | 0     | 0.0  |
| 21    | BldgType     | 0     | 0.0   | 62    | KitchenAbvGr  | 0     | 0.0  |
| 22    | Neighborhood | 0     | 0.0   | 63    | Functional    | 0     | 0.0  |
| 23    | LandSlope    | 0     | 0.0   | 64    | FullBath      | 0     | 0.0  |
| 24    | LotConfig    | 0     | 0.0   | 65    | HalfBath      | 0     | 0.0  |
| 25    | Condition1   | 0     | 0.0   | 66    | GarageCars    | 0     | 0.0  |
| 26    | LandContour  | 0     | 0.0   | 67    | GarageArea    | 0     | 0.0  |
| 27    | LotShape     | 0     | 0.0   | 68    | BedroomAbvGr  | 0     | 0.0  |
| 28    | Street       | 0     | 0.0   | 69    | PavedDrive    | 0     | 0.0  |
| 29    | LotArea      | 0     | 0.0   | 70    | EnclosedPorch | 0     | 0.0  |
| 30    | MSSubClass   | 0     | 0.0   | 71    | 3SsnPorch     | 0     | 0.0  |
| 31    | MSZoning     | 0     | 0.0   | 72    | OpenPorchSF   | 0     | 0.0  |
| 32    | Id           | 0     | 0.0   | 73    | WoodDeckSF    | 0     | 0.0  |
| 33    | Utilities    | 0     | 0.0   | 74    | PoolArea      | 0     | 0.0  |
| 34    | HouseStyle   | 0     | 0.0   | 75    | ScreenPorch   | 0     | 0.0  |

| Index | Feature     | Число | Доля | Index | Feature       | Число | Доля |
|-------|-------------|-------|------|-------|---------------|-------|------|
| 35    | Foundation  | 0     | 0.0  | 76    | MiscVal       | 0     | 0.0  |
| 36    | ExterQual   | 0     | 0.0  | 77    | MoSold        | 0     | 0.0  |
| 37    | ExterCond   | 0     | 0.0  | 78    | YrSold        | 0     | 0.0  |
| 38    | BsmtUnfSF   | 0     | 0.0  | 79    | SaleType      | 0     | 0.0  |
| 39    | TotalBsmtSF | 0     | 0.0  | 80    | SaleCondition | 0     | 0.0  |
| 40    | Heating     | 0     | 0.0  | 81    | SalePrice     | 0     | 0.0  |
| 41    | BsmtFinSF1  | 0     | 0.0  | -     | -             | -     | -    |

В таблице столбец **Число** означает число пропусков, а столбец **Доля** означает долю пропусков в процентах. Названия сокращены, чтобы таблица не обрезалась по ширине.

В таблице выше все столбцы расставлены по убыванию числа пропусков. В столбце **LotFrontage** пропусков всего 259.

Границу для столбцов с большим числом пропусков мы взяли по содержанию этой таблицы, а именно **выше 47 %** (так охватываются, по сути, все доли пропусков из двух цифр)

Это сделано потому, что пропуски в переменной **LotFrontage** будут заменены средним.

```
# какие столбцы содержат пропусков больше, чем 47 % всех наблюдений
too_many_null_cols = null_count_by_col[null_count_by_col > 0.47 * len(df)].index
joined = ', '.join(too_many_null_cols.astype(str))
print("Столбцы, где пропусков больше 47% всех наблюдений "
      f"{joined[:len(joined)//2]} \n {joined[len(joined)//2:]}" )
# берём join, чтобы соединить строки в одну и поставить между
# ними разделитель, то есть запятую
```

Столбцы, где пропусков больше 47% всех наблюдений Alley , MasVnrType , FireplaceQu , PoolQC , Fence , MiscFeature

```
# удаляем все найденные выше столбцы, где пропусков больше 47%
df.drop(columns=too_many_null_cols, inplace=True)
```

```
# отдельно посмотрим на нужный нам столбец LotFrontage,
# возьмём только первые двадцать значений
col_LotFrontage = df["LotFrontage"].head(20)
type_list = []
for T in col_LotFrontage:
    # добавляем в список только название типа
    type_list.append(type(T).__name__)

# делаем из первых двадцати значений LotFrontage отдельную таблицу,
```

```
# которая является копией и не связана с исходным df
df_LF = df[["LotFrontage"]].head(20).copy()
# добавляем сведения о типе прямо рядом с прежним столбцом
df_LF["Type"] = type_list

# убираем запись вида ".0", столбец Type при этом трогать нельзя
df_LF["LotFrontage"] = pd.to_numeric(df_LF["LotFrontage"],
                                     errors="coerce").astype("Int64")
total_row_num = len(df_LF)
half_row_num = (total_row_num + 1) // 2

left = df_LF.iloc[:half_row_num].reset_index(drop = True)
right = df_LF.iloc[half_row_num:].reset_index(drop = True)

merged = pd.concat([left, right], axis = 1)
HTML(merged.to_html(index = False, border = 1))
```

| LotFrontage | Type  | LotFrontage | Type  |
|-------------|-------|-------------|-------|
| 65          | float | 70          | float |
| 80          | float | 85          | float |
| 68          | float | <NA>        | float |
| 60          | float | 91          | float |
| 84          | float | <NA>        | float |
| 85          | float | 51          | float |
| 75          | float | <NA>        | float |
| <NA>        | float | 72          | float |
| 51          | float | 66          | float |
| 50          | float | 70          | float |

**Значение <NA> (= пропуск ) тоже распознаётся как тип float. Отсюда видно, что по типу судить о пропуске нельзя.**

```
# убираем ненужный столбец Id, как того требует условие задания
if "Id" in df.columns:
    df.drop(columns=["Id"], inplace = True)
if "Id" in df.columns:
    print("Столбец Id удалён не полностью")
else:
    print("Столбец Id удалён полностью")
```

Столбец Id удалён полностью

```
# отделяем признаки от целевой переменной
feature_df = df.drop(columns = ["SalePrice"])
target_series = df["SalePrice"]
# делим данные на обучающие и проверочные в отношении 8 к 2,
# как того требует условие задания
X_train, X_test, y_train, y_test = train_test_split(feature_df, target_series,
                                                    test_size = 0.2,
                                                    random_state = 42)
```

```
# работа с пропусками в LotFrontage, среднее считаем только
# по обучающей части и одно и то же значение применяем к обеим частям
if "LotFrontage" in X_train.columns:
    # среднее посчитано только на обучающих данных
    lf_mean = X_train["LotFrontage"].mean()
    X_train["LotFrontage"] = X_train["LotFrontage"].fillna(lf_mean)
    X_test["LotFrontage"] = X_test["LotFrontage"].fillna(lf_mean)

# превращаем качественные признаки в набор из нулей и единиц,
# причём делаем это после разделения, отдельно для каждой части
X_train = pd.get_dummies(X_train, drop_first = True)
X_test = pd.get_dummies(X_test, drop_first = True)

# приводим столбцы проверочной части к столбцам обучающей,
# а недостающие заполняем нулями
X_test = X_test.reindex(columns = X_train.columns, fill_value = 0)
```

```
# проверим, хорошо ли убрались пропуски в LotFrontage
print("Число пропусков LotFrontage в обучающей части ",
      X_train["LotFrontage"].isna().sum())
print("""*30)
print("Число пропусков LotFrontage в проверочной части ",
      X_test["LotFrontage"].isna().sum())
```

Число пропусков LotFrontage в обучающей части 0

Число пропусков LotFrontage в проверочной части 0

**Здесь может возникнуть вопрос, почему замена средним сделана уже после разделения на обучающую и проверочную части.**

Причина в том, что если посчитать среднее до разделения, то в него попадут те значения **LotFrontage** из проверочной части, которые пропусками не являются, и само среднее из-за этого может измениться. Если заменять пропуски таким средним, то оно применяется и к обучающей, и к проверочной части. Значит, **в ходе подготовки данных сведения из проверочной части могут попасть в расчёт среднего, и в итоге качество на проверочной части может выйти лучше, чем оно есть на самом деле.** ( Например, MAE и RMSE могут получиться меньше, а  $R^2$  больше )

**Поэтому ради более строгой оценки среднее считалось только по обучающим данным, и то же самое значение применялось к проверочным, что здесь и оговаривается.**

```
# проверим, хорошо ли прошла работа с пропусками
col_deleted = ["Alley", "MasVnrType", "FireplaceQu", "PoolQC", "Fence",
               "MiscFeature"]

# 1. проверяем, удалились ли столбцы с большим числом пропусков (выше 47 %)
still_exists = [c for c in col_deleted if c in df.columns]
if len(still_exists) == 0:
    print("Все шесть столбцов с долей пропусков выше 47% удалены")
else:
    print("Остались неудалённые столбцы ", still_exists)
```

Все шесть столбцов с долей пропусков выше 47% удалены

```
# 2. проверяем, исчезли ли пропуски в столбце LotFrontage
# у X_train и у X_test
if "LotFrontage" not in X_train.columns:
    print("В X_train нет столбца LotFrontage")
else:
    n_missing_train = X_train["LotFrontage"].isna().sum()
    n_missing_test = X_test["LotFrontage"].isna().sum()

    if (n_missing_train == 0) and (n_missing_test == 0):
        print("Пропуски LotFrontage исчезли в обеих частях")
    else:
        print(f"Пропусков в обучающей {n_missing_train} "
              f"| в проверочной {n_missing_test}")
```

Пропуски LotFrontage исчезли в обеих частях

```
print("Пропуски (NaN) в X обучающей части ", X_train.isna().sum().sum())  
print("Пропуски (NaN) в X проверочной части ", X_test.isna().sum().sum())
```

Пропуски (NaN) в X обучающей части 70

Пропуски (NaN) в X проверочной части 19

**Здесь может возникнуть вопрос, куда делись те многочисленные пропуски, которые должны были остаться после всей проделанной работы.**

Дело в том, что если внимательно посмотреть на сведения о пропусках по всем переменным, приведённые на странице 3, видно, что многие пропуски, которые должны были дожить до этого шага, исчезли...! Если говорить точнее, по строкам с номерами Index от 8 до 19 на странице 3 пропусков было в сумме 601, а сейчас их осталось всего 89 ( = 70 в обучающей и 19 в проверочной ).

⇒ **Чтобы понять, почему так вышло, посмотрим на тип данных у всех столбцов, где были пропуски.**

```

# берём три столбца с наибольшим числом пропусков, по убыванию
train_top3 = X_train.isna().sum().sort_values(ascending=False).head(3)
test_top3 = X_test.isna().sum().sort_values(ascending=False).head(3)

# левая часть ( X_train ), это признак и число пропусков
left = pd.DataFrame({"Признак (обучение)": train_top3.index.astype(str),
                    "Число пропусков": train_top3.values.astype(int)})

# правая часть ( X_test ), это признак и число пропусков
right = pd.DataFrame({"Признак (проверка)": test_top3.index.astype(str),
                    "Число пропусков": test_top3.values.astype(int)})

# ставим левую и правую части рядом, по три строки в каждой,
# и получаем таблицу из трёх строк и четырёх столбцов
merged = pd.concat([left.reset_index(drop=True),
                    right.reset_index(drop=True)], axis=1)
merged.columns = ["Признак (обучение)", "Число пропусков",
                  "Признак (проверка)", "Число пропусков"]

HTML(merged.to_html(index=False, border=1))

```

| Признак (обучение) | Число пропусков | Признак (проверка) | Число пропусков |
|--------------------|-----------------|--------------------|-----------------|
| GarageYrBlt        | 64              | GarageYrBlt        | 17              |
| MasVnrArea         | 6               | MasVnrArea         | 2               |
| MSSubClass         | 0               | MSSubClass         | 0               |

**Что же за пропуски уцелели после всей проделанной работы по удалению и замене**

**и остались в столбцах **GarageYrBlt** и **MasVnrArea** ?**

```

# берём из прежней таблицы только столбцы, где есть хотя бы один
# пропуск, и расставляем их по убыванию числа пропусков
# пишем эту строку заново, чтобы код читался проще
null_count_by_col = df.isnull().sum()
# условие вынесено отдельно ради наглядности кода
condition = (null_count_by_col > 0) & (null_count_by_col.index != "LotFrontage")
all_null_col = null_count_by_col[condition].sort_values(ascending=False)
# столбец LotFrontage мы уже сознательно заполнили средним,
# поэтому он сюда не входит

```

```

# создаём таблицу, где есть только признак и тип переменной
dtype_map = df.dtypes.astype(str)
html_table = pd.DataFrame({"Feature": all_null_col.index,
                           "Тип": dtype_map.loc[all_null_col.index].values})

# считаем число строк, чтобы разделить таблицу на левую и правую части
total_row_num = len(html_table)
half_row_num = (total_row_num + 1) // 2
left = html_table.iloc[:half_row_num].reset_index(drop=True)
right = html_table.iloc[half_row_num:].reset_index(drop=True)

# добавляем столбец Index в каждую часть, нумерация идёт с единицы
# и продолжается сквозной по всей таблице
left.insert(0, "Index", range(1, len(left) + 1))
right.insert(0, "Index", range(len(left) + 1, len(left) + len(right) + 1))

# закрепляем у Index целый тип Int64 и убираем запись вида ".0"
for side_table in (left, right):
    side_table["Index"] = pd.to_numeric(side_table["Index"],
                                         errors="coerce").astype("Int64")

# ставим левую и правую части рядом по горизонтали
merged = pd.concat([left, right], axis=1)

# правая часть короче, и появившиеся из-за этого NaN заменяем знаком "-"
merged_display = merged.astype(object).where(merged.notna(), "-")
HTML(merged_display.to_html(index=False, border=1))

```

| Index | Feature      | Тип     | Index | Feature      | Тип     |
|-------|--------------|---------|-------|--------------|---------|
| 1     | GarageYrBlt  | float64 | 7     | BsmtExposure | object  |
| 2     | GarageFinish | object  | 8     | BsmtQual     | object  |
| 3     | GarageQual   | object  | 9     | BsmtFinType1 | object  |
| 4     | GarageType   | object  | 10    | BsmtCond     | object  |
| 5     | GarageCond   | object  | 11    | MasVnrArea   | float64 |
| 6     | BsmtFinType2 | object  | 12    | Electrical   | object  |

Прежде всего, есть сильное основание полагать, что пропуски у **качественных** ( **categorical** ) переменных вроде **GarageCond** / **GarageType** / **BsmtQual** / **Electrical** уже превратились в *get\_dummies* из NaN в « наборы нулей и единиц ».

А пропуски остались только в столбцах **GarageYrBlt** и **MasVnrArea** потому, что это **числовые** ( **numeric** ) переменные.

**Опираясь на сведения двух предыдущих страниц, разберёмся с оставшимися пропусками более толково.**

Проверено, что в итоговых данных уже не осталось пропусков у **качественных** ( **categorical** ) переменных вроде **GarageCond** / **GarageType** / **BsmtQual** / **Electrical**, с которыми мы напрямую не работали, поэтому, учитывая начальный уровень этого задания, дальше мы идём без дополнительной работы с ними.

Однако иначе обстоит дело со столбцами, где лежат **числовые** ( **numeric** ) переменные **GarageYrBlt** и **MasVnrArea**, в них пропуски ещё есть, и поэтому оставшуюся работу мы доводим до конца именно на этих столбцах.

При этом стоит помнить, что переменные **GarageYrBlt** и **MasVnrArea** означают соответственно *год, когда у этого дома впервые был построен гараж*, и *площадь каменной облицовки, то есть отделочного камня на наружной стене здания*. Отсюда следует, что для **GarageYrBlt** разумно заменять пропуски **серединным значением**, ведь надо учитывать и наличие гаража, и то, как распределены годы, а для **MasVnrArea** пропуск, скорее всего, означает отсутствие каменной отделки, и поэтому естественнее всего заменить его на **0**.

**Опираясь на такое рассуждение, работу с пропусками в оставшихся столбцах мы выполнили так, как показано ниже.**

```
# пропуски GarageYrBlt заменяем серединным значением обучающей части
if "GarageYrBlt" in X_train.columns:
    X_train.loc[:, "GarageYrBlt"] = pd.to_numeric(
        X_train["GarageYrBlt"], errors="coerce"
    )
    X_test.loc[:, "GarageYrBlt"] = pd.to_numeric(
        X_test["GarageYrBlt"], errors="coerce"
    )

    median_val = X_train["GarageYrBlt"].median()
    X_train.loc[:, "GarageYrBlt"] = X_train["GarageYrBlt"].fillna(median_val)
    X_test.loc[:, "GarageYrBlt"] = X_test["GarageYrBlt"].fillna(median_val)
```

```
# пропуски MasVnrArea заменяем нулём, ведь это означает отсутствие площади
if "MasVnrArea" in X_train.columns:
    X_train.loc[:, "MasVnrArea"] = pd.to_numeric(
        X_train["MasVnrArea"], errors="coerce"
    ).fillna(0)
    X_test.loc[:, "MasVnrArea"] = pd.to_numeric(
        X_test["MasVnrArea"], errors="coerce"
    ).fillna(0)
```

**Перед самым обучением и разбором можно убедиться, что все пропуски убраны.**

```
print("Пропуски (NaN) в X обучающей части ", X_train.isna().sum().sum())
print("Пропуски (NaN) в X проверочной части ", X_test.isna().sum().sum())
```

Пропуски (NaN) в X обучающей части 0  
Пропуски (NaN) в X проверочной части 0

**Разбор проводится на основе всей проделанной до этого подготовки данных.**

```
# обучаем регрессионное дерево, а именно DecisionTreeRegressor,
# как того требует условие задания
tree_regressor = DecisionTreeRegressor(random_state=42)
tree_regressor.fit(X_train, y_train)
# получаем предсказания на проверочных данных и считаем качество
test_pred = tree_regressor.predict(X_test)
mae_value = mean_absolute_error(y_test, test_pred)
mse_value = mean_squared_error(y_test, test_pred)
rmse_value = np.sqrt(mse_value)
r2_value = r2_score(y_test, test_pred)
# выводим оценки качества
print(f"Оценка модели | MAE ( средняя абсолютная ошибка ) : {mae_value:.2f}")
print(f"Оценка модели | MSE ( средняя квадратичная ошибка ) : {mse_value:.2f}")
print(f"Оценка модели | RMSE ( корень из средней квадратичной ) : {rmse_value:.2f}")
print(f"Оценка модели | R2 ( доля объяснённой изменчивости ) : {r2_value:.2f}")
```

Оценка модели | MAE ( средняя абсолютная ошибка ) : 29070.50  
Оценка модели | MSE ( средняя квадратичная ошибка ) : 1885684420.92  
Оценка модели | RMSE ( корень из средней квадратичной ) : 43424.47  
Оценка модели | R2 ( доля объяснённой изменчивости ) : 0.75

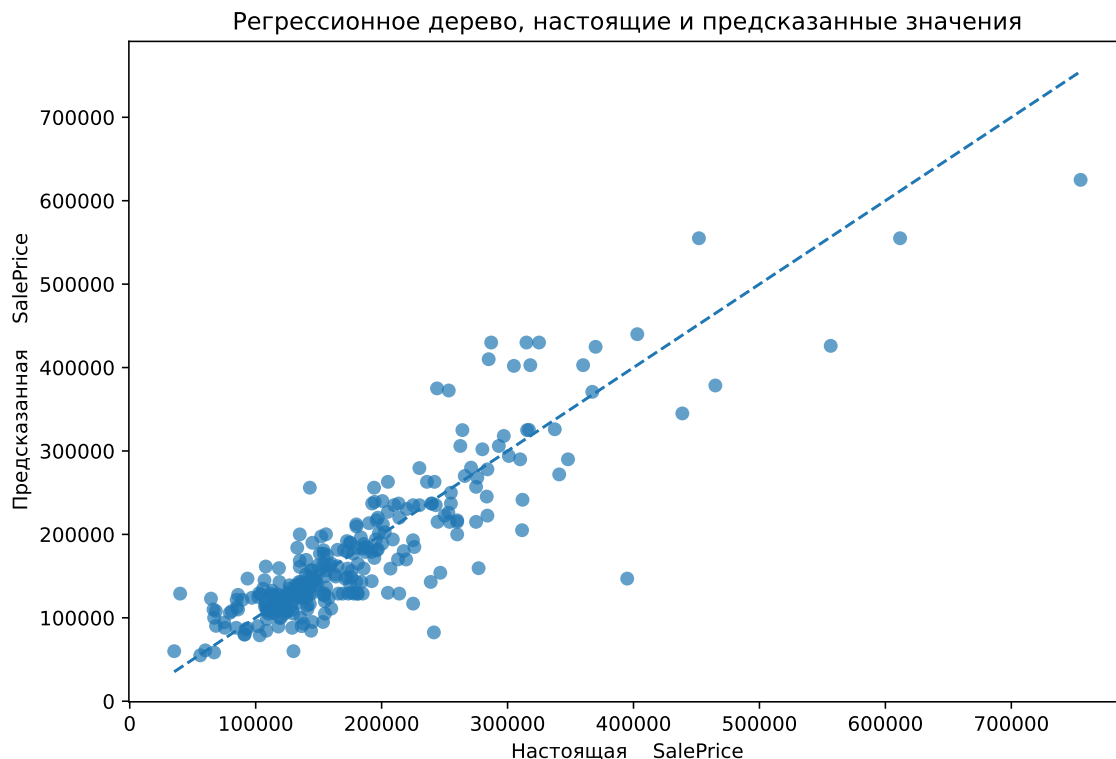
**Приведённые выше результаты подробно разбираются дальше, на странице 12.**

```
import matplotlib.pyplot as plt

# рисунок с результатами обучения, для многомерной регрессии
# это обычное сравнение настоящих значений с предсказанными
# шрифт с поддержкой кириллицы, чтобы буквы не пропадали
plt.rc('font', family = "DejaVu Sans")

plt.figure(figsize = (9, 6))
# alpha делает точки темнее там, где они накладываются друг на друга
plt.scatter(y_test, test_pred, alpha = 0.7)
plt.xlabel("Настоящая SalePrice")
plt.ylabel("Предсказанная SalePrice")
plt.title("Регрессионное дерево, настоящие и предсказанные значения")

# опорная линия y = x, при безошибочном предсказании
# все точки лежали бы прямо на ней
min_v = min(y_test.min(), test_pred.min())
max_v = max(y_test.max(), test_pred.max())
plt.plot([min_v, max_v], [min_v, max_v], linestyle="--")
plt.show()
```



Если показать результаты разбора на рисунке, как сделано выше, видно, что настоящие и предсказанные цены жилья в целом лежат вблизи наклонной линии ( $y = x$ ). Отсюда же

видно, что общий уровень цен модель улавливает сравнительно неплохо. Однако чем ближе к области дорогого жилья, тем сильнее точки отходят от этой линии, то есть в высоком ценовом промежутке ошибка предсказания заметно растёт, и в этом состоит ограничение модели.

**Толкование этого приведено в пункте (3) на следующей странице.**

# Толкование результатов

Если учитывать смысл задания, а именно « **подготовить данные о ценах жилья, обучить регрессионную модель на дереве решений и оценить качество предсказания** », то приведённое ниже **толкование результатов** тоже оказывается осмысленным.

- (1) Прежде всего, при подготовке данных столбцы были отобраны по числу пропусков, и переменные, где пропусков больше 47 % всех наблюдений, были удалены. Это можно считать разумным шагом, поскольку учитывается то, что **переменные с большой долей пропусков способны сделать обучение модели неустойчивым**. Кроме того, у переменной LotFrontage пропуски были заменены средним значением, что уменьшило их остаток, а у переменных GarageYrBlt и MasVnrArea пропуски были восполнены соответственно срединным значением и нулём, с оглядкой на смысл самих переменных, и тем самым возможные ошибки из-за пропусков были заранее и естественным образом сняты. Качественные переменные были преобразованы через `get_dummies ( drop_first = True )`, и это указывает на то, что данные приведены к виду, при котором модель может обучаться на одних только числовых входных значениях.
- (2) Далее, при оценке качества регрессионной модели на дереве решений **MAE составила 29070.50**, а значит, на проверочных данных предсказание расходится с настоящей ценой жилья в среднем примерно на 2,9 десятка тысяч долларов по модулю. При этом RMSE оказалась равна **43424.47**, то есть заметно больше MAE, и это, в силу самой природы показателей, основанных на квадрате ошибки, указывает на возможность сравнительно больших ошибок на части наблюдений. Наконец, значение  $R^2$  составило **0.75**, а значит, эта модель объясняет примерно 75 % изменчивости цен жилья на проверочных данных, и это численно подтверждает, что даже после простой подготовки данных дерево решений даёт определённую предсказательную силу.
- (3) Вместе с тем, если посмотреть на рисунок с настоящими и предсказанными значениями, видно **нарастание ошибки прежде всего в промежутке дорогого жилья**. Так происходит потому, что очень дорогого жилья в данных сравнительно мало, и модели трудно набрать достаточно примеров для обучения, а дерево решений по своей сути собирает похожие наблюдения вместе и предсказывает по их среднему. В итоге в области дорогого жилья похожих наблюдений не хватает, свойство дерева усреднять проявляется сильнее, и в нескольких местах предсказание выходит ниже настоящей цены. Это указывает на то, что в среднем и низком ценовом промежутке, где обучающих данных сравнительно много, ошибка выходит небольшой, тогда как в промежутке очень дорогого жилья, где наблюдений мало, сведений для обучения не хватает и ошибка растёт.

В целом эти результаты показывают, что весь порядок работы, при котором **удаляются переменные с большой долей пропусков, ключевые пропуски восполняются средним, качественные переменные переводятся в числа, а затем качество регрессионной модели оценивается числами**, даёт в задаче предсказания цен жилья **сразу и осмысленную объясняющую силу ( $R^2 = 0.75$ ), и понятный уровень ошибки (MAE и RMSE)**. В частности, MAE равна 29 070,50, и это численно показывает, что **средняя ошибка предсказания составляет примерно 2,9 десятка тысяч долларов**, а то, что RMSE равна 43 424,47 и превышает MAE, указывает на **возможность сравнительно больших ошибок на отдельных участках (RMSE чутка к большим ошибкам)}**. Кроме того,  $R^2 = 0.75$  означает, что модель **объясняет примерно 75 % изменения цен на проверочных данных, а оставшиеся 25 % изменения это то, что нынешняя модель уловила недостаточно**. Значит, эта модель обладает начальной предсказательной силой, но **по величине ошибки место для дальнейшего уточнения ещё остаётся**.

Этот документ подготовлен в формате **Quarto Markdown**(.qmd), который позволяет объединять код **Python** и текст документа в одном файле, и выведен в формате PDF.